



Boletín 01 Intermedio (Resuelto) — Algorítmica y Pseudocódigo

Soluciones completas, razonadas y verificadas en pseudocódigo PSeInt para los 10 ejercicios del Boletín Intermedio de la Unidad 1.

A continuación se detallan las soluciones exhaustivas de los 10 ejercicios planteados en el [Boletín Intermedio](#).

Solución al Ejercicio 1: Validación estricta de entrada con bucle Repetir

```
Algoritmo CalificacionCualitativa
  Definir nota Como Real

  // Filtro de validación mediante bucle con post-condición
  Repetir
    Escribir "Introduce la nota académica del alumno [0.0 - 10.0]:"
    Leer nota
    Si nota < 0 O nota > 10 Entonces
      Escribir "Error: La nota debe encontrarse obligatoriamente entre 0.0 y
10.0."
    FinSi
  Hasta Que nota ≥ 0 Y nota ≤ 10

  // Clasificación cualitativa mediante condicionales anidadas
  Si nota < 5.0 Entonces
    Escribir "Calificación obtenida: Suspenso"
  SiNo
    Si nota < 6.0 Entonces
      Escribir "Calificación obtenida: Aprobado"
    SiNo
      Si nota < 7.0 Entonces
        Escribir "Calificación obtenida: Bien"
      SiNo
        Si nota < 9.0 Entonces
          Escribir "Calificación obtenida: Notable"
        SiNo
          Escribir "Calificación obtenida: Sobresaliente"
        FinSi
      FinSi
    FinSi
  FinSi
FinAlgoritmo
```

Solución al Ejercicio 2: Bucle con centinela: Contador de pares e impares

```

Algoritmo ContadorParesImparesCentinela
  Definir n, pares, impares Como Entero

  pares ← 0
  impares ← 0

  Escribir "Introduce números enteros (introduce 0 para finalizar):"
  Leer n

  // El bucle Mientras se ejecuta hasta que el usuario introduzca el centinela 0
  Mientras n ≠ 0 Hacer
    Si n MOD 2 = 0 Entonces
      pares ← pares + 1
    SiNo
      impares ← impares + 1
    FinSi

    Escribir "Siguiendo número (0 para terminar):"
    Leer n
  FinMientras

  Escribir "===== "
  Escribir "  RECuento FINAL DE DATOS  "
  Escribir "===== "
  Escribir "Total de números pares:   ", pares
  Escribir "Total de números impares: ", impares
FinAlgoritmo

```

Solución al Ejercicio 3: Búsqueda de mayor y menor de 5 números sin arrays

```

Algoritmo MayorYMenorDeCinco
  Definir i, n, mayor, menor Como Entero

  Escribir "Introduce el 1º número:"
  Leer n

  // ¡CRÍTICO! Inicializamos mayor y menor con el PRIMER valor real introducido.
  // Si hubiéramos puesto "mayor ← 0" y todos los números fuesen negativos (-10, -5,
  // -20),
  // el algoritmo diría falsamente que el mayor es 0 cuando el usuario jamás lo
  introdujo.
  mayor ← n
  menor ← n

  Para i ← 2 Hasta 5 Con Paso 1 Hacer
    Escribir "Introduce el ", i, "º número:"
    Leer n

    Si n > mayor Entonces
      mayor ← n
    FinSi

    Si n < menor Entonces
      menor ← n
    FinSi
  FinPara

  Escribir "_____ "
  Escribir "El número MAYOR introducido es: ", mayor
  Escribir "El número MENOR introducido es: ", menor
FinAlgoritmo

```

Solución al Ejercicio 4: Subalgoritmo Factorial iterativo y análisis de traza

1. Código PSeInt del subalgoritmo y algoritmo principal:

```

Funcion resultado ← factorial(n)
  Definir resultado, i Como Entero
  resultado ← 1
  Para i ← 1 Hasta n Con Paso 1 Hacer
    resultado ← resultado × i
  FinPara
FinFuncion

Algoritmo PruebaFactorial
  Definir num Como Entero
  Escribir "Introduce un entero positivo para calcular su factorial:"
  Leer num
  Si num ≥ 0 Entonces
    Escribir num, "! = ", factorial(num)
  SiNo
    Escribir "El factorial no está definido para números negativos."
  FinSi
FinAlgoritmo

```

2. Tabla de traza para la invocación `factorial(4)` :

Ciclo	Variable <code>i</code>	Operación realizada	Valor de <code>resultado</code>
Inicio	<i>indefinido</i>	<code>resultado ← 1</code>	1
Iteración 1	1	<code>resultado ← 1 × 1</code>	1
Iteración 2	2	<code>resultado ← 1 × 2</code>	2
Iteración 3	3	<code>resultado ← 2 × 3</code>	6
Iteración 4	4	<code>resultado ← 6 × 4</code>	24
Fin bucle	—	Retorno del subalgoritmo	24

Solución al Ejercicio 5: Introducción a la recursividad (`factorialRecursivo`)

```

Funcion resultado ← factorialRecursivo(n)
  Definir resultado Como Entero

  // CASO BASE: Detiene la recursión cuando n alcanza 0 o 1
  Si n ≤ 1 Entonces
    resultado ← 1
  SiNo
    // CASO RECURSIVO: n! = n × (n - 1)!
    resultado ← n × factorialRecursivo(n - 1)
  FinSi
FinFuncion

```

Comparativa teórica (Iterativo vs Recursivo):

- **Legibilidad y elegancia:** La versión recursiva es prácticamente idéntica a la definición matemática formal ($n! = n \times (n-1)!$). Una vez dominada la recursividad, resulta conceptualmente muy limpia.
- **Consumo de memoria:** La versión iterativa (`Para`) utiliza un único espacio de memoria fijo. La versión recursiva crea un **marco de pila (stack frame)** en la memoria RAM por cada llamada activa, esperando a que los casos más pequeños retornen su valor. Para valores grandes de `n`, una función recursiva sin optimizar puede desbordar la pila (*StackOverflowError*).

Solución al Ejercicio 6: Diseño modular integral: Calculadora de IMC con validación

```
// 1. Subalgoritmo para pedir y validar el peso
SubProceso p ← pedirPeso()
  Definir p Como Real
  Repetir
    Escribir "Introduce el peso en kilogramos (ejemplo: 72.5):"
    Leer p
    Si  $p \leq 0$  Entonces
      Escribir "Error: El peso debe ser un número estrictamente positivo."
    FinSi
  Hasta Que  $p > 0$ 
FinSubProceso

// 2. Subalgoritmo para pedir y validar la altura
SubProceso a ← pedirAltura()
  Definir a Como Real
  Repetir
    Escribir "Introduce la altura en metros (ejemplo: 1.75):"
    Leer a
    Si  $a \leq 0$  O  $a > 2.80$  Entonces
      Escribir "Error: La altura debe ser válida y positiva [0.5 - 2.80 m]."a > 0 Y  $a \leq 2.80$ 
FinSubProceso

// 3. Algoritmo principal coordinador
Algoritmo CalculoIMCModular
  Definir peso, altura, imc Como Real

  Escribir "≡ SISTEMA CLÍNICO DE CÁLCULO DE IMC ≡"
  peso ← pedirPeso()
  altura ← pedirAltura()

   $imc \leftarrow peso / (altura \times altura)$ 

  Escribir "_____ "
  Escribir "Tu Índice de Masa Corporal (IMC) es: ", imc

  Si  $imc < 18.5$  Entonces
    Escribir "Diagnóstico OMS: Bajo peso (insuficiencia ponderal)."imc < 25.0 Entonces
      Escribir "Diagnóstico OMS: Intervalo normal / Peso saludable."
    SiNo
      Si  $imc < 30.0$  Entonces
        Escribir "Diagnóstico OMS: Sobrepeso."
      SiNo
        Escribir "Diagnóstico OMS: Obesidad."
      FinSi
    FinSi
  FinSi
FinSi
FinAlgoritmo
```

Solución al Ejercicio 7: Comprobador de números primos optimizado (`esPrimo`)

```

Funcion esP ← esPrimo(n)
  Definir esP Como Logico
  Definir divisor Como Entero

  Si n < 2 Entonces
    esP ← Falso
  SiNo
    esP ← Verdadero
    divisor ← 2
    // Optimización matemática: Si no tiene divisores hasta su raíz cuadrada, es
    primo
    Mientras divisor × divisor ≤ n Y esP = Verdadero Hacer
      Si n MOD divisor = 0 Entonces
        esP ← Falso // Divisor exacto encontrado: no es primo
      FinSi
      divisor ← divisor + 1
    FinMientras
  FinSi
FinFuncion

Algoritmo ProbarPrimalidad
  Definir num Como Entero
  Escribir "Introduce un número entero:"
  Leer num
  Si esPrimo(num) Entonces
    Escribir "El número ", num, " es PRIMO."
  SiNo
    Escribir "El número ", num, " es COMPUESTO (no es primo)."
  FinSi
FinAlgoritmo

```

Solución al Ejercicio 8: Contador y sumador de dígitos de un número entero

```

Algoritmo ContadorYSumadorDigitos
  Definir numOriginal, numCopia, ultimoDigito, totalDigitos, sumaDigitos Como Entero

  Escribir "Introduce un número entero positivo:"
  Leer numOriginal

  Si numOriginal < 0 Entonces
    numCopia ← -numOriginal // Tomamos valor absoluto si fuese negativo
  SiNo
    numCopia ← numOriginal
  FinSi

  totalDigitos ← 0
  sumaDigitos ← 0

  Si numCopia = 0 Entonces
    totalDigitos ← 1
    sumaDigitos ← 0
  SiNo
    Mientras numCopia > 0 Hacer
      ultimoDigito ← numCopia MOD 10          // Extraemos el dígito de las
unidades
      sumaDigitos ← sumaDigitos + ultimoDigito
      totalDigitos ← totalDigitos + 1
      numCopia ← trunc(numCopia / 10)          // Recortamos el último dígito
    FinMientras
  FinSi

  Escribir "El número ", numOriginal, " tiene ", totalDigitos, " dígitos."
  Escribir "La suma de todos sus dígitos es: ", sumaDigitos
FinAlgoritmo

```

Solución al Ejercicio 9: Determinación de Año Bisiesto (Regla Gregoriana)

```

Algoritmo AnioBisiestoGregoriano
  Definir anio Como Entero
  Definir esBisiesto Como Logico

  Escribir "Introduce el año a comprobar:"
  Leer anio

  // Regla gregoriana unificada:
  // Divisible por 4 Y NO por 100, O BIEN divisible por 400
  esBisiesto ← (anio MOD 4 = 0 Y anio MOD 100 ≠ 0) O (anio MOD 400 = 0)

  Si esBisiesto Entonces
    Escribir "El año ", anio, " es BISIESTO (febrero tiene 29 días, total 366
días)."
  SiNo
    Escribir "El año ", anio, " es COMÚN (febrero tiene 28 días, total 365 días)."
  FinSi
FinAlgoritmo

```

Solución al Ejercicio 10: Diagnóstico de errores y arquitectura de la JVM

1. Diagnóstico y clasificación de errores:

- (a) Olvidar una palabra reservada como `FinSi` o un punto y coma:
 - **Tipo:** Error **Sintáctico** (de compilación).
 - **Momento de detección:** En tiempo de análisis léxico/sintáctico por parte del compilador o del intérprete antes de iniciar la ejecución del programa. El compilador rechaza el archivo y reporta la línea y columna del fallo gramatical.
- (b) Calcular el área de un círculo usando la fórmula del perímetro ($2 \times PI \times r$):
 - **Tipo:** Error **Semántico o Lógico** (*bug*).
 - **Momento de detección:** Durante la fase de pruebas (QA) o en producción. El programa compila sin advertencias y se ejecuta con total normalidad, pero el valor numérico obtenido es incorrecto. Requiere depuración manual o trazado sobre papel.
- (c) Dividir entre cero cuando la variable tiempo vale `0`:
 - **Tipo:** Error **en Tiempo de Ejecución** (*Runtime Exception*).
 - **Momento de detección:** En pleno funcionamiento del programa, en el instante exacto en que la CPU intenta ejecutar la operación aritmética inválida. Provoca el cierre abrupto de la aplicación si no se gestiona mediante control de excepciones (`try-catch`).

2. Ventajas del modelo híbrido de Java:

- **Frente a un lenguaje compilado nativo como C:** Ofrece **portabilidad total** (*Write Once, Run Anywhere*). En C, el código binario generado está atado a la arquitectura de CPU (x86, ARM) y al sistema operativo; en Java, el mismo fichero binario intermedio (`.class` o `.jar`) se ejecuta idénticamente en cualquier plataforma sin necesidad de recompilar el proyecto, requiriendo únicamente la JVM de dicho entorno.
- **Frente a un lenguaje puramente interpretado como Python:** Ofrece **mayor rendimiento y verificación temprana**. Al compilarse previamente a Bytecode, se descartan los errores sintácticos y de tipado antes de la ejecución. Además, el motor **JIT** de la JVM compila los métodos más ejecutados a código máquina nativo ultrarrápido en caliente, alcanzando velocidades cercanas a las de C++.